



International Islamic University Chittagong

Department of Computer Science and Engineering

Lab Report

Lab No 4: Implementation of Polynomial Circle Algorithm

Course Title: Computer Graphics Lab

Course Code: CSE-4742

Session: Autumn 2026

Semester: 7th

Section: 7BF

Student Name	Tahasina Tasnim Afra
Student ID	C233456
Department	Computer Science and Engineering

Submitted To

Mrs. Sultana Tasnim
Lecturer, Department of CSE, IIUC

Submission Date:

September 12, 2026

1. Number & Name of the Experiment

Lab 4: Implementation of Polynomial circle algorithm

2. Introduction

The Polynomial Circle Algorithm is a circle scan-conversion method used in computer graphics to generate a circle by calculating its pixel coordinates from the circle equation. For a circle with center (h, k) and radius r , the equation is

$$(x - h)^2 + (y - k)^2 = r^2.$$

For a circle centered at $(0, 0)$, this becomes

$$x^2 + y^2 = r^2.$$

The algorithm calculates the points for one portion of the circle and uses eight-way symmetry to generate the complete circle. The coordinates are obtained using the square-root form of the circle equation.

3. Algorithm

The Polynomial Circle Algorithm calculates the points of a circle using the equation

$$x^2 + y^2 = r^2.$$

Starting from the top of the circle, the algorithm calculates points from 90° to 45° . The remaining points are obtained using eight-way symmetry.

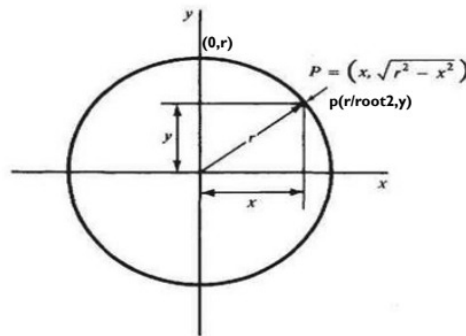


Figure 1: Polynomial method for calculating a circle point

Initial Point

The algorithm starts from the top of the circle:

$$\boxed{x = 0, \quad y = r}$$

For example, if $r = 200$, the initial point is

$$(0, 200).$$

Calculation of y

From

$$x^2 + y^2 = r^2,$$

the value of y is calculated as

$$\boxed{y = \sqrt{r^2 - x^2}}.$$

The value of x is increased incrementally, and the corresponding value of y is calculated for each x .

Stopping Point

Only the portion from the 90° position to the 45° position is calculated. The calculation stops at approximately

$$\boxed{x = \frac{r}{\sqrt{2}}}.$$

At this point,

$$x \approx y \approx \frac{r}{\sqrt{2}}.$$

For $r = 200$, the stopping value is approximately

$$\frac{200}{\sqrt{2}} \approx 141.42.$$

Eight-Way Symmetry

After calculating a point (x, y) , the remaining seven points are obtained using eight-way symmetry. This is possible because a circle has symmetry about the horizontal, vertical, and diagonal axes.

For a circle centered at (h, k) , the eight symmetric points are

$$\begin{array}{ll} (h + x, k + y), & (h + y, k + x), \\ (h - y, k + x), & (h - x, k + y), \\ (h - x, k - y), & (h - y, k - x), \\ (h + y, k - x), & (h + x, k - y). \end{array}$$

Implementation

The main steps of the algorithm are:

1. Set the radius r and initialize $x = 0$.
2. Calculate the ending value $x = r/\sqrt{2}$.
3. Calculate y using

$$y = \sqrt{r^2 - x^2}.$$

4. Round x and y to obtain pixel coordinates.
5. Plot the eight symmetric points using `putpixel()`.
6. Increase x by one.
7. Repeat until $x > r/\sqrt{2}$.

The `PolynomialCircle()` function performs the main calculation, while the eight symmetric points are plotted around the center of the graphics window. The screen coordinates are adjusted because the screen y -coordinate increases downward.

The calculated (x, y) coordinates are also displayed using `cout`.

4. Program with Screenshot

Code Screenshot

The C++ implementation of the Polynomial Circle Algorithm can be found in the following GitHub repository: <https://github.com/afrahackerman/AFRA-comp-graphics>.

The following screenshot shows the implementation:

```

1  #include <graphics.h>
2  #include <cmath>
3  #include <iostream>
4  #include <conio.h>
5  using namespace std;
6  // $file="C233456_polycircle"; g++ "$file.cpp" -o "$file" -lbg -lmdi32 -lcomdlg32 -luuid -oleaut32 -ole32
7  void PolynomialCircle(int r)
8  {
9      float x = 0;
10     float i = 1;
11     float x_end = r / sqrt(2.0);
12
13     int centerX = 400;
14     int centerY = 300;
15
16     while (x <= x_end)
17     {
18         float y = sqrt((r * r) - (x * x));
19         int px = round(x);
20         int py = round(y);
21
22         putpixel(centerX + px, centerY - py, WHITE);
23         putpixel(centerX + py, centerY - px, WHITE);
24         putpixel(centerX - py, centerY - px, WHITE);
25         putpixel(centerX - px, centerY - py, WHITE);
26
27         putpixel(centerX - px, centerY + py, WHITE);
28         putpixel(centerX - py, centerY + px, WHITE);
29         putpixel(centerX + py, centerY + px, WHITE);
30         putpixel(centerX + px, centerY + py, WHITE);
31
32         cout << "x = " << px << ", y = " << py << endl;
33         x = x + i;
34     }
35 }
36
37 int main()
38 {
39     initwindow(800, 600, (char*)"Polynomial Circle Drawing");
40     outtextxy(10, 10, (char*)"C233456");
41     int r = 200;
42     PolynomialCircle(r);
43     getch();
44     closegraph();
45     return 0;
46 }

```

Figure 2: C++ implementation of the Polynomial Circle Algorithm

5. Output/Result

The program was successfully executed and generated a complete circle using the Polynomial Circle Algorithm. The required circle points were calculated from the circle equation and plotted using `putpixel()`. Only one portion of the circle was calculated directly, while eight-way symmetry was used to generate the remaining points.

For the selected radius of 200, the calculation started from the initial point (0, 200) and continued incrementally up to approximately the 45° position, where x and y become nearly equal. The calculated points were then reflected across the different axes to produce the complete circle.

The graphics window also displayed the student ID (C233456), and the resulting circle was clearly visible in the output window.

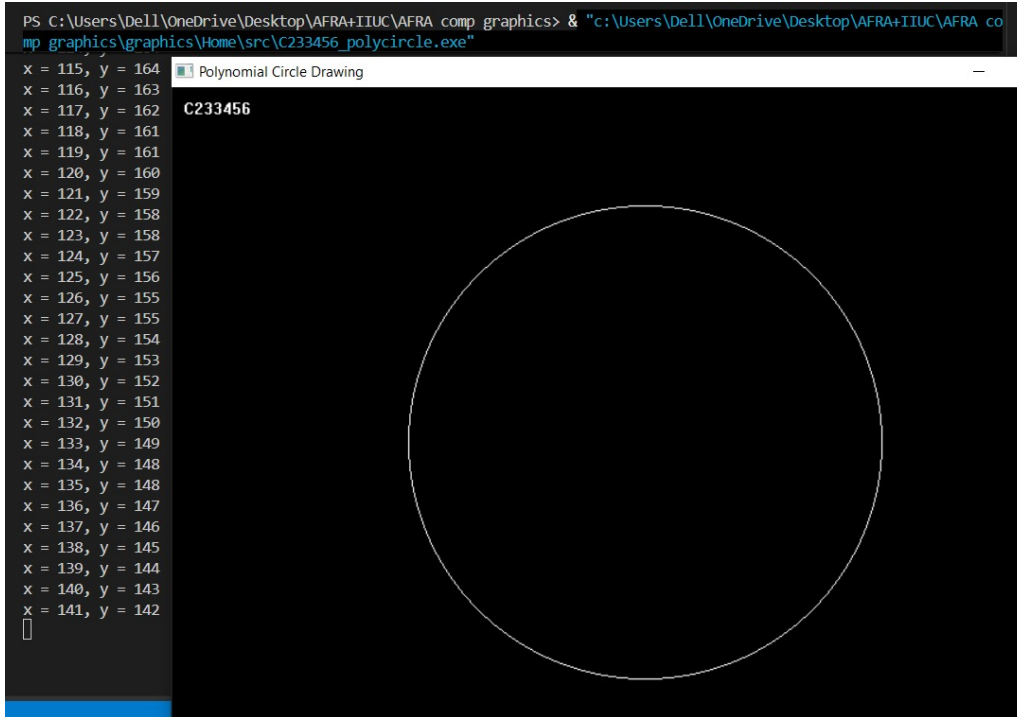


Figure 3: Output of the Polynomial Circle Algorithm

6. Discussion

The Polynomial Circle Algorithm successfully generated a circle by calculating the value of y for each increment of x using

$$y = \sqrt{r^2 - x^2}.$$

Only one portion of the circle was calculated directly, while the remaining points were obtained using eight-way symmetry. This reduced the number of calculations required to generate the complete circle.

The method is simple and directly follows the mathematical equation of a circle. However, it requires square-root and floating-point calculations, making it less efficient than incremental algorithms such as the Midpoint Circle Algorithm.

7. Conclusion

The Polynomial Circle Algorithm was successfully implemented in C++ using the `graphics.h` library. The experiment demonstrated how the circle equation can be used to calculate pixel coordinates and generate a complete circle using eight-way symmetry.

The experiment also provided an understanding of the basic process of circle scan conversion and pixel plotting using `putpixel()`.